

EPHEC — HAUTE ECOLE | TECH

Classe : 2AU — Année académique 2025-2026
A2073 - Robotique (Part 2)

RAPPORT FINAL

Rubik's Cube Solver Robot

Conception et réalisation d'un robot autonome de résolution du Rubik's Cube

Réalisé par :

EL YAZAMI Yassine

MAZID Mohamed

ELHAMAYDA Amer

Déposé le : mercredi 3 juin 2026

Table des matières

Table des matières	2
Introduction	4
1. Partie électronique	5
1.1 Présentation des composants.....	5
1.2 Schéma électrique.....	6
1.3 Conception et routage du PCB	7
2. Partie mécanique	9
2.1 Structure générale	9
2.2 Base découpée au laser	9
2.3 Pièces modélisées en 3D sur Fusion 360.....	10
Support de servomoteur — platine principale.....	10
Support vertical — colonne de bras	10
Pince — bras de saisie	11
Support de caméra	11
Engrenage	12
Cadre de structure	12
3. Partie programmation.....	15
3.1 Architecture logicielle globale	15
3.2 Firmware Raspberry Pi Pico (MicroPython)	16
3.2.1 Organisation des modules.....	16
3.2.2 Boucle principale et gestion du bouton.....	17
3.2.3 Contrôle des servomoteurs — calibrage individuel.....	17
3.2.4 Scan automatique des 6 faces	17
3.3 Application PC (Python 3).....	17
3.3.1 Interface graphique Tkinter	17
3.3.2 Détection des couleurs par caméra (OpenCV/HSV).....	18
3.3.3 Visualisation du cube — CubeView.....	18
3.4 Algorithme de résolution — Kociemba.....	18
4. Difficultés rencontrées.....	20
5. Conclusion générale	21
6. Bibliographie	22
Documentation technique des composants	22
Logiciels et outils de développement	22
Bibliothèques logicielles	22
Algorithme de résolution.....	23
7. Annexes	24
Annexe A — Liste complète des composants.....	24
Annexe B — Répartition des tâches	24
Annexe C — Diagramme de Gantt	25

Annexe D — Informations PCB 25

Annexe E — Démonstration vidéo..... 26

Introduction

Le Rubik's Cube est un puzzle inventé en 1974 par l'architecte hongrois **Ernő Rubik**. Avec ses **43 quintillions** de configurations possibles, ses **6 faces** et ses **26 pièces mobiles**, il représente un vrai défi, autant sur le plan mathématique qu'algorithmique. En 2010, des chercheurs ont montré que n'importe quelle position du cube peut être résolue en **20 mouvements maximum**.

C'est dans ce contexte que s'inscrit notre projet de deuxième année dans le cadre du cours **A2073 – Robotique**. L'objectif était de concevoir et de réaliser un robot capable de résoudre automatiquement un Rubik's Cube, sans intervention humaine, en combinant trois domaines complémentaires : la mécanique, l'électronique embarquée et le développement logiciel.

Le système final mis au point par notre équipe repose sur une architecture en deux parties. D'un côté, un **Raspberry Pi Pico** équipé d'un **PCB** conçu sur mesure pilote **huit servomoteurs** répartis en quatre bras, lit un badge **RFID**, affiche des informations sur un écran **OLED** et communique sans fil grâce à un module **Bluetooth HC-05**. De l'autre, une application **Python** exécutée sur PC s'occupe de capturer et d'analyser les couleurs du cube à l'aide d'une caméra, de calculer la solution avec l'algorithme **Kociemba**, puis de piloter l'ensemble du processus via une interface graphique **Tkinter**.

Ce rapport final présente tout le travail réalisé au cours de l'année académique **2025-2026**. Il est organisé en trois grandes parties techniques — **électronique, mécanique et programmation** — chacune suivie d'une conclusion partielle. L'ensemble se termine par un bilan des difficultés rencontrées et une conclusion générale.

1. Partie électronique

1.1 Présentation des composants

L'ensemble de l'électronique du robot repose sur un PCB sur mesure, conçu avec le logiciel EasyEDA et fabriqué via JLCPCB. Ce PCB mesure 81,15 mm × 88,14 mm et intègre 21 composants répartis sur 2 couches de cuivre, avec 141 pastilles traversantes et 6 vias. Le tableau suivant détaille les composants utilisés.

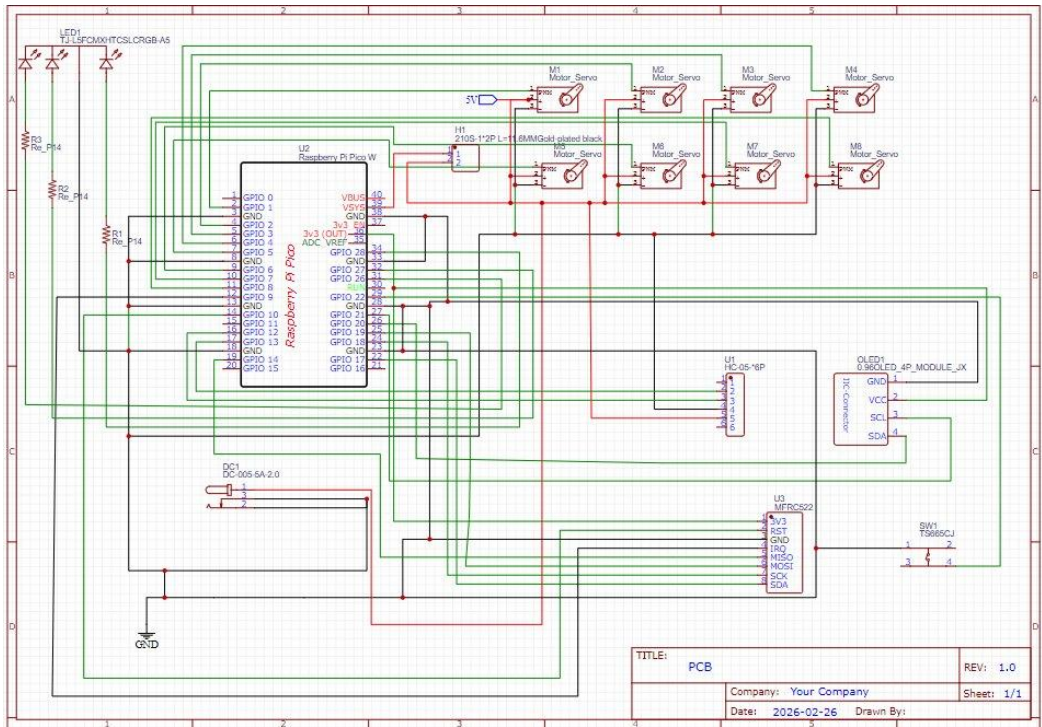


Figure 1 — Liste des composants générée depuis EasyEDA / JLCPCB

Composant	Référence	Rôle dans le système
Raspberry Pi Pico W	RP2040	Microcontrôleur principal — firmware MicroPython
Module Bluetooth	HC-05	Communication sans fil PC — PICO (UART, 38 400 bauds)
Lecteur RFID	MFRC522	Identification par badge — autorisation d'accès
Ecran OLED	SSD1306 0,96"	Affichage état du robot, face scannée, progression
LED RGB	TJ-L5FCMXHTCSLCRGB	Indicateur visuel d'état (rouge = bloqué, vert = prêt, bleu = actif)
Servomoteurs	MG90S x8	4 rotations + 4 pinces — pilotés par PWM 50 Hz — 1,8 kg/cm à 5V
Bouton	TS665CJ	Appui court : toggle pinces — Appui long : scan auto

Connecteur alimentation	DC-005-5A-2.0	Alimentation externe 5 V pour les servomoteurs
Résistances	Re_P14 x3	Protection des lignes de signal GPIO

Tableau 1 — Composants électroniques du système

Le **Raspberry Pi Pico W** a été choisi pour ses nombreuses possibilités de connexion matérielle. Il dispose de deux cœurs **ARM Cortex-M0+** à **133 MHz**, de nombreux **GPIO** pouvant être utilisés en **PWM**, de deux ports **UART**, ainsi que des bus **I2C** et **SPI**. Son support de **MicroPython** facilite le développement et les tests du firmware.

L'alimentation des servomoteurs est séparée de celle du Pico afin d'éviter les baisses de tension qui peuvent apparaître lorsque plusieurs moteurs fonctionnent en même temps.

1.2 Schéma électrique

Le schéma électrique ci-dessous a été réalisé sous EasyEDA. Il représente l'ensemble des connexions entre le Raspberry Pi Pico W et les composants périphériques : les huit servomoteurs (M1 à M8), le module Bluetooth HC-05 (U1), l'écran OLED (via connecteur I2C), le lecteur RFID MFRC522 (U3), la LED RGB (LED1), le bouton (SW1) et le connecteur d'alimentation DC (DC1).

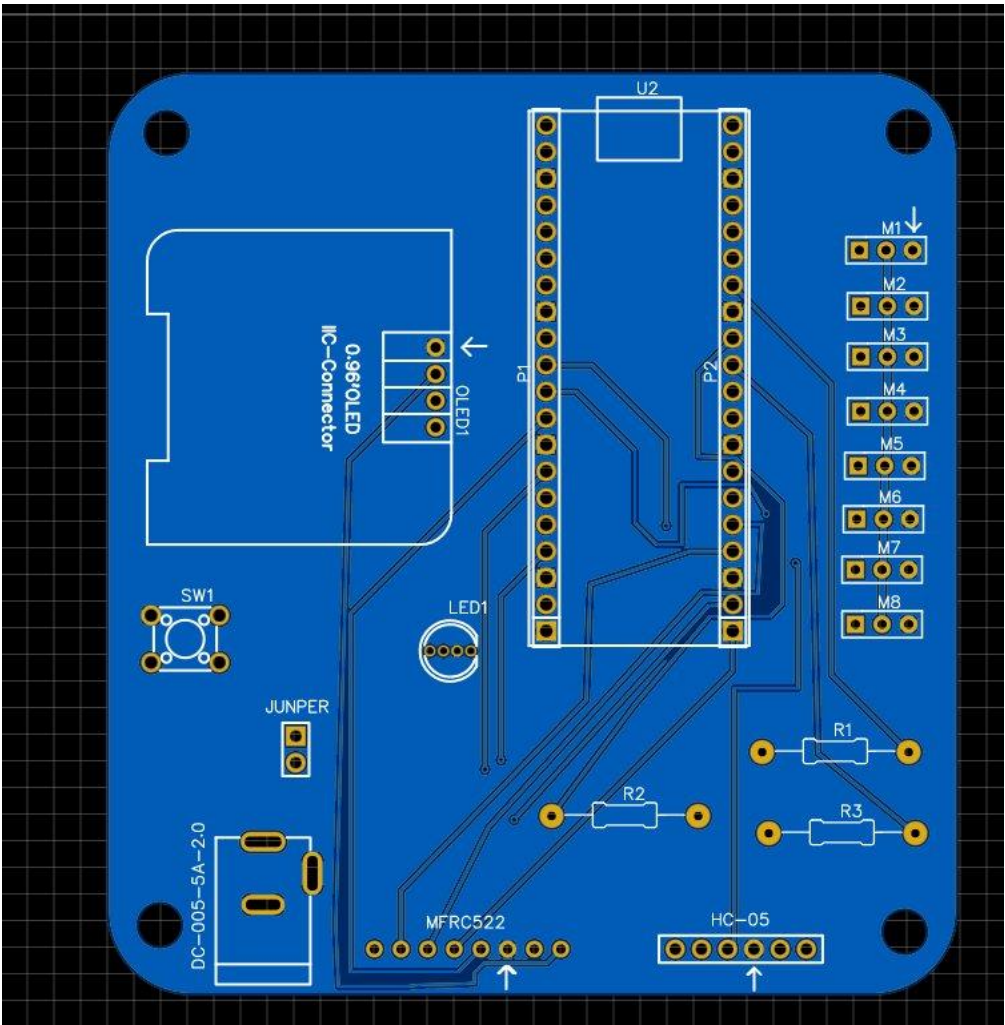


Figure 2 — Schéma électrique complet du PCB (EasyEDA, révision 1.0 — 26/02/2026)

Les interfaces de communication utilisées sont les suivantes :

- UART 0 (GPIO 12 TX / GPIO 13 RX, buffer 512 octets) : communication Bluetooth avec le HC-05 à 38 400 bauds
- SPI 0 (GPIO 18 SCK / GPIO 19 MOSI / GPIO 0 MISO / GPIO 17 CS / GPIO 10 RST) : lecteur RFID MFRC522
- I2C 0 (GPIO 20 SDA / GPIO 21 SCL) : écran OLED SSD1306 128x64 pixels
- PWM sur GPIO 1-8 : pilotage des 8 servomoteurs (fréquence 50 Hz)
- PWM sur GPIO 26, 27, 2 : LED RGB (rouge, vert, bleu)
- GPIO 22 (entrée pull-up) : bouton physique

1.3 Conception et routage du PCB

Le routage du **PCB** a été réalisé sous **EasyEDA** en tenant compte de la place disponible et de l'organisation des différents composants. Les connecteurs des huit servomoteurs (**M1 à M8**) ont été regroupés sur le côté droit de la carte afin de faciliter le câblage vers les bras du robot. Le module **HC-05** et le lecteur **MFRC522** sont placés dans la partie inférieure du PCB pour rendre leurs connexions plus accessibles. Les résistances de protection **R1**, **R2** et **R3** ont été installées au plus près des GPIO concernés.



Figure 3 — Vue de routage du PCB (gauche) et rendu 3D vue de dessus avec composants (droite)

Information sur le PCB	
Taille:	81.15mm x 88.14mm
Couches de signaux:	2
Couches sans signaux:	10
Composants:	21
Pastilles:	141
Surface Pads:	0
Plated Through-hole Pads:	141
None Plated Through-hole Pads:	0
Trous:	4
Vias:	6
Nets:	81/81
Longueur des pistes:	1554.74mm
Zones de cuivre:	1

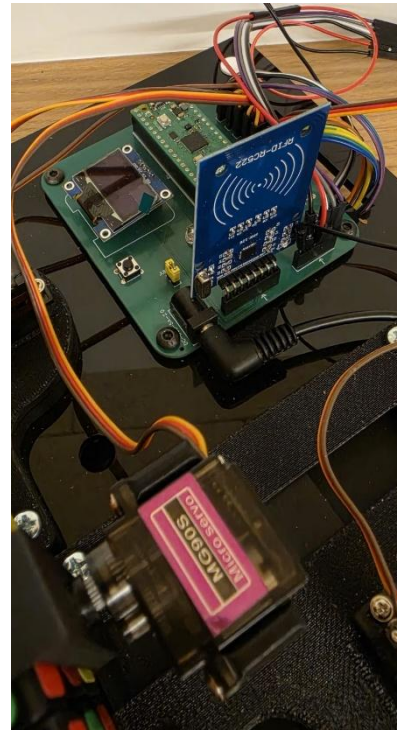


Figure 5 — PCB physique après soudure et assemblage

Conclusion partielle — Partie électronique

La partie électronique du projet a abouti à la conception et à la fabrication d'un PCB sur mesure de 81,15 mm × 88,14 mm, intégrant 21 composants sur 2 couches. Le Raspberry Pi Pico W centralise l'ensemble des interfaces matérielles nécessaires au projet : PWM pour les servomoteurs, UART pour le Bluetooth, SPI pour le RFID et I2C pour l'OLED. La séparation de l'alimentation des servomoteurs et du microcontrôleur garantit la stabilité du système lors des mouvements. Les tests unitaires réalisés composant par composant ont validé le bon fonctionnement de chaque module avant l'intégration finale.

2. Partie mécanique

2.1 Structure générale

La structure mécanique du robot est organisée autour d'une base centrale sur laquelle viennent se fixer quatre bras disposés en croix, chacun orienté vers l'une des quatre faces latérales du cube (Front, Back, Left, Right). Chaque bras est composé d'un servomoteur de rotation et d'un servomoteur de pince, ce qui permet de saisir et de faire tourner le cube selon les axes nécessaires à la résolution. La conception de toutes les pièces a été réalisée avec le logiciel Fusion 360.

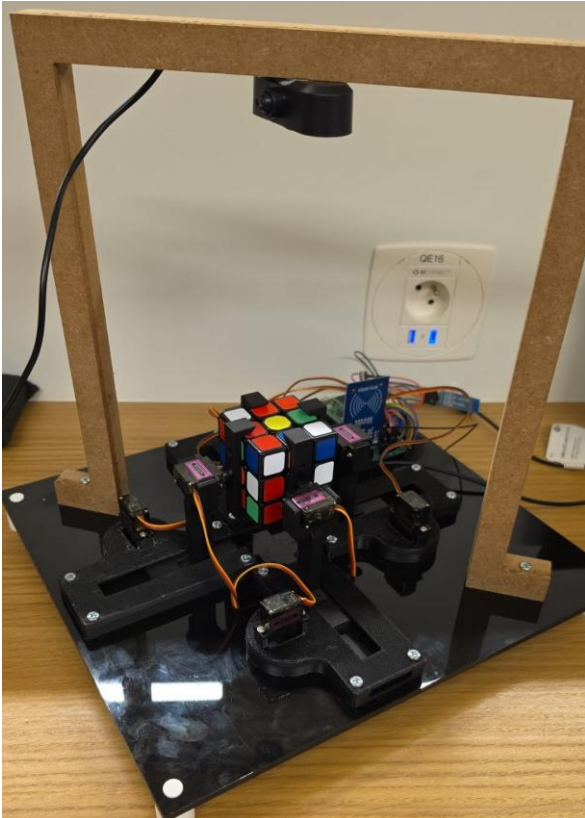


Figure 6

2.2 Base découpée au laser

La base du robot est une plaque découpée au laser portant l'inscription « Rubik's Cube Solveur — Robotique A2073 — 2025/2026 ». Elle comporte quatre emplacements symétriques destinés à recevoir les bras du robot, avec des encoches de fixation permettant un positionnement précis. Des trous de fixation aux quatre coins permettent de stabiliser l'ensemble sur la surface de travail.

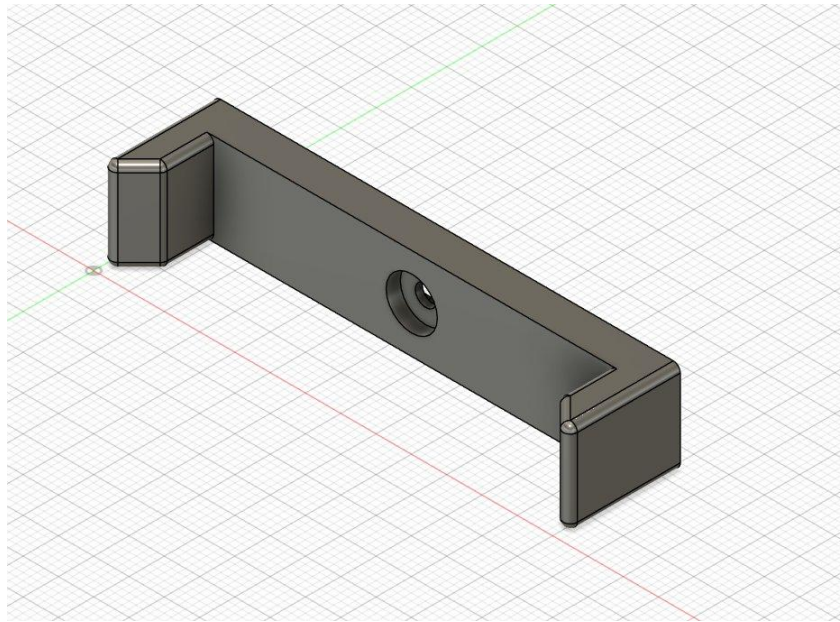


Figure 7 — Plan de découpe laser de la base du robot (fusion / vue CAO)

2.3 Pièces modélisées en 3D sur Fusion 360

L'ensemble des pièces constitutives des bras a été modélisé en 3D sous Fusion 360 et imprimé en PLA. Chaque bras est un assemblage de plusieurs pièces distinctes, dont les rôles sont décrits ci-dessous. Plusieurs itérations d'impression ont été nécessaires afin d'ajuster les tolérances dimensionnelles et d'obtenir des mouvements précis et sans jeu excessif.

Support de servomoteur — platine principale

La platine principale constitue la base de chaque bras. Elle accueille le servomoteur de rotation et assure la fixation sur la base laser. Sa forme intègre les dégagements nécessaires à l'axe du servomoteur et les trous de fixation permettant un montage rigide.

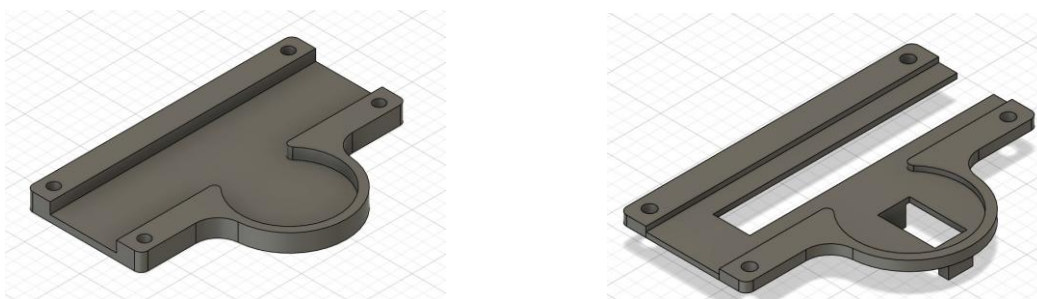


Figure 8 — Platine support du servomoteur de rotation : version pleine (gauche) et avec dégagement axe (droite)

Support vertical — colonne de bras

La colonne verticale supporte le servomoteur de pince à la hauteur du cube. Elle est fixée sur la platine principale et maintient le servomoteur de pince en position haute, de manière à ce que la pince soit alignée avec les faces du cube positionné au centre du robot.

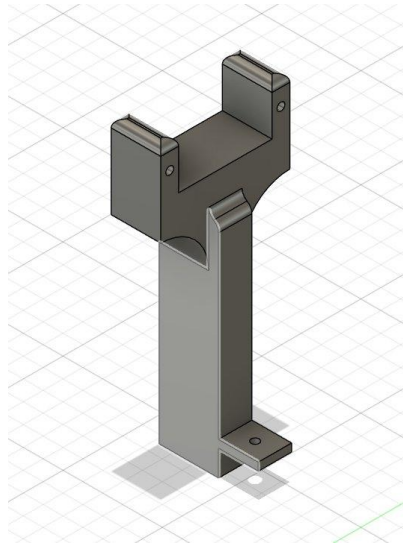


Figure 9 — Colonne verticale du bras avec support de pince

Pince — bras de saisie

La pince est l'élément qui entre directement en contact avec le cube. Elle est actionnée par le servomoteur de pince et alterne entre deux positions : serrée (saisie du cube) et ouverte (relâche). Sa forme en U permet d'envelopper une face du cube sans glisser lors des rotations.

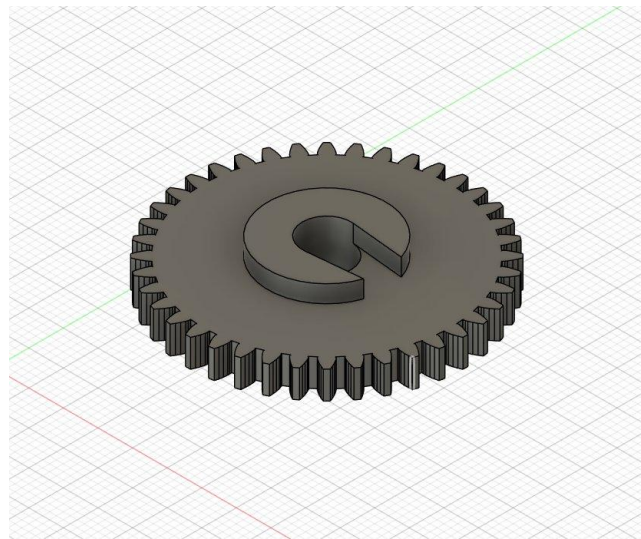


Figure 10 — Pièce de pince en U — saisie des faces du cube

Support de caméra

Le support de caméra est la pièce qui maintient la caméra USB en position fixe au-dessus du centre du robot. Sa forme rectangulaire avec encoches latérales permet un positionnement stable et ajustable en hauteur. La lumière centrale guide le câble de la caméra et l'orifice de fixation assure un maintien rigide pendant les mouvements du robot.

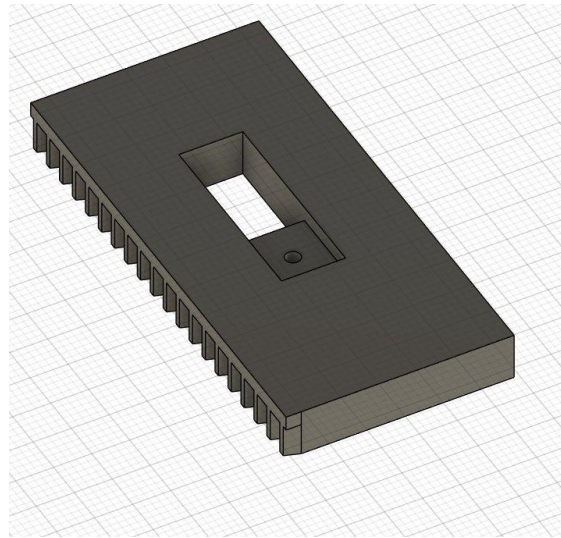


Figure 11 — Support de caméra (modélisation Fusion 360)

Engrenage

Un engrenage a été modélisé pour assurer la transmission de mouvement dans les configurations nécessitant un rapport de démultiplication ou un changement d'axe. Le trou central en D garantit l'enclenchement sur l'axe du servomoteur.

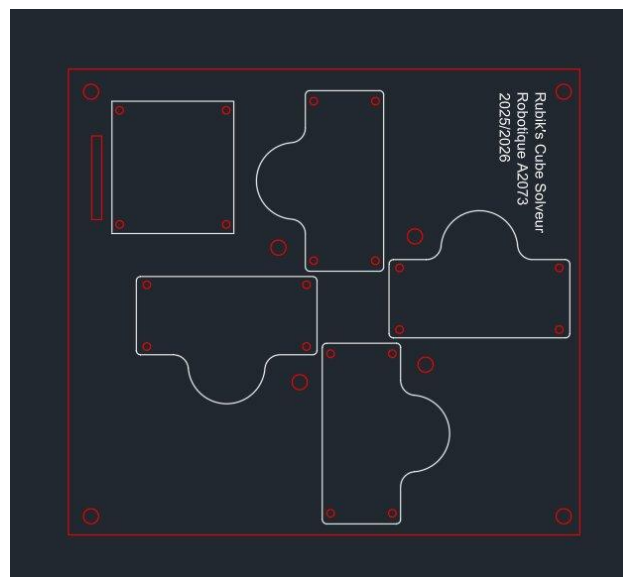


Figure 12 — Engrenage avec trou en D pour axe servomoteur

Cadre de structure

Le cadre en U constitue l'ossature externe de chaque bras. Il relie la platine principale à la colonne verticale et offre un support rigide à l'ensemble du mécanisme. Ses dimensions ont été ajustées pour s'adapter à la taille standard d'un Rubik's Cube 3x3.

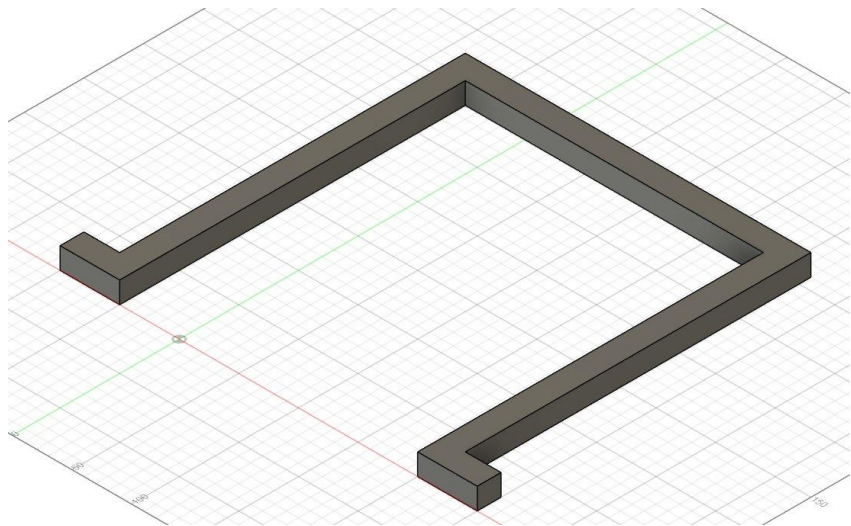


Figure 13 — Cadre structural en U du bras

Caractéristique	Valeur (4,8V)	Valeur (6V)
Poids	13,4 g	—
Dimensions	22,5 × 12 × 35,5 mm	—
Couple de blocage	1,8 kg/cm	2,2 kg/cm
Vitesse de rotation (60°)	0,10 s	0,08 s
Tension de fonctionnement	4,8 V — 6,6 V	—
Signal PWM	50 Hz — impulsion 1 à 2 ms	—
Type d'engrenages	Métal	—

Tableau 2 — Caractéristiques techniques du servomoteur MG90S

Paramètre	Bras L	Bras R / F	Bras B
ROT_CENTER (position neutre)	86°	90°	94°
GRIP_HOLD (saisie)	20°	18° / 30°	35°
GRIP_OPEN (relâche)	100°	100°	100°
TEMPS_MVT (résolution)	0,12 s	0,12 s	0,12 s
TEMPS_SCAN (scan faces)	0,30 s	0,30 s	0,30 s

Tableau 3 — Paramètres de calibrage par bras (version finale)

Conclusion partielle — Partie mécanique

La conception mécanique a été l'aspect le plus itératif du projet. La découpe laser a fourni une base précise et reproductible, tandis que la modélisation Fusion 360 a permis de

concevoir et d'ajuster les pièces des bras avant chaque impression. Plusieurs révisions ont été nécessaires pour corriger les tolérances et obtenir des mouvements fiables. La calibration individuelle de chaque bras (ROT_CENTER différent pour L, R, F et B) a permis de compenser les légères asymétries mécaniques et d'obtenir une précision de positionnement satisfaisante.

3. Partie programmation

3.1 Architecture logicielle globale

Le système logiciel est divisé en deux blocs indépendants qui coopèrent via une liaison Bluetooth : le firmware embarqué sur le Raspberry Pi Pico, développé en MicroPython, et l'application de contrôle tournant sur PC, développée en Python 3 avec l'environnement de développement VS Code.

Les deux parties communiquent via un protocole maison structuré sous la forme :

START : TAG : PAYLOAD : END\n

Ce format permet d'identifier chaque type de message et de le distinguer facilement du bruit ou des messages d'initialisation émis spontanément par le module HC-05. Un buffer logiciel côté Pico (rxbuf=512 octets) absorbe les longues solutions Kociemba sans perte de données.

L'algorithme ci-dessous présente le fonctionnement général du système, côté PC (gauche) et côté Raspberry Pi Pico (droite). Il illustre les deux flux parallèles et leurs points de synchronisation via Bluetooth.

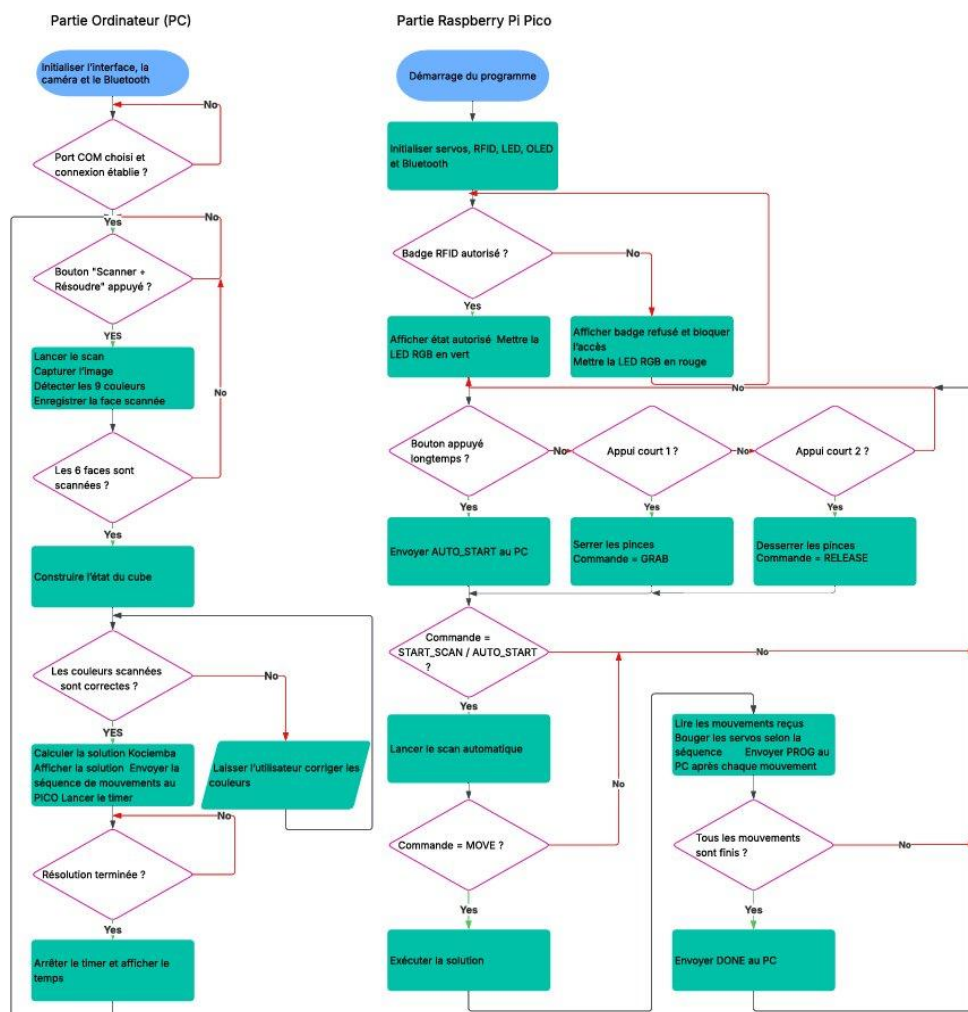


Figure 14 — Algorithmme général du système — Partie PC (gauche) et Partie PICO (droite)

Direction	Tag	Description
PC -> PICO	MOVE	Séquence de mouvements à exécuter (ex: R U R' F2)
PC -> PICO	GRAB / RELEASE	Serrer ou relâcher les 4 pinces
PC -> PICO	START_SCAN	Déclencher le scan automatique des 6 faces
PC -> PICO	SCAN_ACK:face	Acquittement : face capturée avec succès
PICO -> PC	SCAN_READY:face	Face positionnée devant la caméra
PICO -> PC	SCAN_DONE:OK	Toutes les 6 faces ont été scannées
PICO -> PC	PROG:i/total	Progression : i-ème mouvement sur total
PICO -> PC	DONE:MOVE	Exécution de la séquence terminée
PICO -> PC	AUTH:id,0 1	Résultat de lecture RFID (0=refusé, 1=autorisé)

Tableau 3 — Messages du protocole Bluetooth START:TAG:PAYLOAD:END

3.2 Firmware Raspberry Pi Pico (MicroPython)

3.2.1 Organisation des modules

Le firmware est organisé en huit modules distincts, chacun responsable d'une fonctionnalité précise, ce qui facilite la maintenance et les tests individuels.

Module	Auteur	Rôle
main.py	Amer	Boucle principale : RFID, bouton, Bluetooth, séquences
servo_controller.py	Amer	Gestion des 4 bras (8 servos) avec calibrage individuel par bras
servo.py	Amer	Classe bas niveau PWM 50 Hz — conversion angle vers duty cycle
state.py	Amer	Machine à états : IDLE, MOVING, SCAN
led_controller.py	Amer	LED RGB par PWM (pins 26, 27, 2) — indication visuelle état
bluetooth.py	Yassine	UART 0 (TX:12, RX:13) — buffer logiciel, parsing START:TAG:PLD:END
rfid_reader.py	Yassine	MFRC522 via SPI 0 — lecture badge, liste blanche, acquittement
oled_display.py	Yassine	SSD1306 via I2C 0 (SDA:20, SCL:21) — 4 lignes de 16 caractères

Tableau 4 — Modules du firmware PICO avec auteur

3.2.2 Boucle principale et gestion du bouton

La boucle principale du fichier `main.py` gère trois flux en simultané : la scrutation RFID toutes les 500 ms, la lecture du bouton physique et la réception de commandes Bluetooth. Le bouton dispose de deux comportements selon la durée d'appui :

- Appui court (< 800 ms) : bascule l'état des pinces entre ouvert et serré, permettant de positionner manuellement le cube
- Appui long (> 800 ms) : déclenche la séquence de scan automatique `auto_scan()`, à condition que le badge RFID ait été préalablement validé

Le système RFID utilise une liste blanche (`ALLOWED_TAGS = [54206395]`). Si le badge présenté n'est pas autorisé, toutes les commandes sensibles sont bloquées et un message d'erreur est envoyé au PC (tag `ERR:NOT_AUTHORIZED`). La LED rouge s'allume et l'OLED affiche un message de refus.

3.2.3 Contrôle des servomoteurs — calibrage individuel

La classe `RobotController` gère les quatre bras (F, B, L, R), chacun instancié comme un objet `Bras` avec ses propres constantes de calibrage. Cette approche par dictionnaire permet de compenser les légères différences mécaniques entre les bras sans modifier la logique de contrôle.

Chaque mouvement du Rubik's Cube (R, U', F2...) est traduit par la fonction `executer_mouvement_rubiks()` en une séquence d'appels servo : rotation CW ou CCW, relâche pince, retour centre, saisie. Les mouvements U et D, qui n'ont pas de bras dédié, sont réalisés par une combinaison de basculements du cube suivis d'une rotation de la face accessible. La résolution utilise `TEMPS_MVT = 0,12 s` par étape, tandis que le scan utilise `TEMPS_SCAN = 0,30 s` pour plus de stabilité.

3.2.4 Scan automatique des 6 faces

La fonction `auto_scan()` orchestre l'exposition de chacune des 6 faces du cube devant la caméra selon une séquence précise de basculements. Pour chaque face, le Pico envoie `SCAN_READY:face` et attend un acquittement `SCAN_ACK:face` du PC dans un délai maximal de 60 secondes. La LED passe en bleu pendant toute la durée du scan et l'OLED affiche la progression (Scan 1/6, Scan 2/6...).

3.3 Application PC (Python 3)

3.3.1 Interface graphique Tkinter

L'application PC est développée par Yassine et propose une interface graphique sombre (thème dark navy, `BG=#1a1a2e`) organisée en plusieurs zones : barre de connexion Bluetooth (port COM, baudrate), contrôles principaux (scan, résolution, envoi), flux caméra en temps réel, vue en croix des 6 faces du cube, panneau solution Kociemba et chronomètre. L'interface est responsive avec une taille minimale de 1200×720 pixels.

3.3.2 Détection des couleurs par caméra (OpenCV/HSV)

La détection des couleurs, développée par Mohamed, est réalisée en espace HSV via OpenCV. Pour chaque case du cube, un patch de 17×17 pixels est analysé et la couleur médiane est comparée à des plages HSV prédéfinies. L'ordre de vérification est crucial : l'orange est testé avant le rouge car leurs plages HSV sont adjacentes.

Couleur	H min	H max	Conditions supplémentaires
Blanc	0	180	S < 60 et V > 170
Orange	6	22	S > 100 et V > 80 — testé avant le rouge
Jaune	23	38	S > 80 et V > 80
Vert	39	85	S > 70 et V > 50
Bleu	95	135	S > 70 et V > 50
Rouge	0-5 / 170-179	—	S > 90 et V > 50 — double plage HSV circulaire

Tableau 5 — Plages de détection HSV par couleur (orange testé avant rouge)

Pour valider une face, la fonction `snapshot_face()` prend 7 captures successives et applique un vote majoritaire case par case : chaque case reçoit la couleur qui a obtenu le plus de votes sur les 7 échantillons. Ce mécanisme rend la détection robuste aux variations d'éclairage ponctuelles. Si une case reste indéterminée sur tous les échantillons, la face est rejetée.

3.3.3 Visualisation du cube — CubeView

Le widget `CubeView`, développé par Mohamed, est un canvas Tkinter qui affiche les 6 faces du cube en disposition en croix (U en haut, L-F-R-B au centre, D en bas). Chaque case est cliquable, ce qui permet de corriger manuellement une couleur mal détectée par un cycle entre les 6 couleurs possibles. Le widget se met à jour en temps réel pendant la résolution simulée.

3.4 Algorithme de résolution — Kociemba

La résolution est assurée par l'algorithme de Herbert Kociemba, intégré localement via le dossier `twophase/` qui contient les tables de recherche précalculées. L'algorithme fonctionne donc hors ligne, sans dépendance à un service externe.

Une fois les 6 faces scannées, la fonction `build_kociemba_string()` construit une chaîne de 54 caractères : elle identifie chaque couleur à la face dont le centre porte cette couleur, puis concatène les 54 cases dans l'ordre U-R-F-D-L-B. La bibliothèque `kociemba.solve()` retourne la séquence optimale en notation standard (R, U', F2...).

```
# Construction de la chaîne + résolution cube_string = build_kociemba_string(faces) #  
ex: 'UUUUUUUUURRRRRR...' solution = kociemba.solve(cube_string) # ex: "R U R' F2 L  
D ..."
```

La solution est ensuite transmise au Pico via Bluetooth (MOVE:R U R' F2...). Le Pico exécute chaque mouvement séquentiellement et renvoie PROG:i/total après chaque étape. L'interface PC met à jour la barre de progression et simule simultanément la résolution sur le CubeView grâce au module CubeState.

Conclusion partielle — Partie programmation

L'architecture logicielle modulaire adoptée, avec une séparation nette entre le firmware PICO et l'application PC, a permis un développement en parallèle efficace entre les membres de l'équipe. Le protocole Bluetooth START:TAG:PAYLOAD:END s'est révélé robuste et simple à implémenter des deux côtés, avec une gestion propre du bruit d'initialisation du HC-05. La détection couleur par vote majoritaire sur 7 captures offre une fiabilité supérieure aux approches à seuil fixe. L'algorithme Kociemba embarqué localement garantit une résolution optimale en 20 mouvements maximum, pour un temps d'exécution physique de l'ordre de 20 secondes avec TEMPS_MVT = 0,12 s.

4. Difficultés rencontrées

La réalisation de ce projet a mis en lumière plusieurs défis techniques, organisationnels et pratiques que notre équipe a dû identifier et surmonter au fil des semaines de développement.

Difficulté	Solution apportée	Résultat
Précision des angles de servomoteurs : certains bras manquaient leur position cible, provoquant des blocages mécaniques	Calibrage individuel par bras via les dictionnaires ROT_CENTER, GRIP_HOLD — chaque bras possède ses propres constantes	Résolu — mouvements reproductibles
Confusion orange/rouge lors de la détection HSV, particulièrement sous éclairage artificiel variable	L'orange est vérifié avant le rouge dans la fonction detect_color(). Ajout d'un fallback par distance circulaire sur la teinte H	Résolu
Instabilité de la détection couleur : résultats différents d'une capture à l'autre pour la même face	Remplacement du système de seuil par un vote majoritaire sur 7 captures (snapshot_face) — plus robuste aux variations ponctuelles	Résolu
Bruit d'initialisation du HC-05 : messages parasites ('OK\r\n') corrompaient le parsing du protocole	Buffer logiciel côté PICO : le code cherche le pattern START: dans le flux et ignore tout ce qui précède. rxbuf=512 pour les longues solutions	Résolu
Chutes de tension lors des mouvements simultanés de plusieurs servomoteurs	Alimentation séparée des servomoteurs (DC-005-5A-2.0). Séquencement des mouvements avec délais pour lisser les pics de courant	Résolu
Tolérances d'impression 3D : encoches trop serrées ou trop lâches pour les servomoteurs	Révisions des fichiers Fusion 360 avec ajustement dimensionnel (+0,2 mm sur les encoches) et réimpression des pièces concernées	Résolu

Tableau 6 — Difficultés rencontrées et solutions mises en place

Au-delà des aspects techniques, la coordination entre les trois membres de l'équipe sur des domaines différents (mécanique, électronique, programmation) a nécessité une organisation rigoureuse. Les dépendances entre les parties — notamment entre le firmware PICO et l'application PC — ont imposé de définir clairement les interfaces (protocole Bluetooth, format des messages) avant de commencer le développement, afin que chaque membre puisse travailler indépendamment.

5. Conclusion générale

Ce projet de robotique constitue l'aboutissement d'une année de travail pluridisciplinaire. L'objectif initial — concevoir et réaliser un robot capable de scanner, résoudre et exécuter la solution d'un Rubik's Cube de manière entièrement autonome — a été atteint.

Sur le plan électronique, le PCB conçu sous EasyEDA et fabriqué via JLCPCB offre une intégration compacte et fiable de l'ensemble des composants. Le Raspberry Pi Pico W s'est révélé être un choix pertinent grâce à la diversité de ses interfaces et à la facilité de développement offerte par MicroPython.

Sur le plan mécanique, la structure en croix avec base laser et bras imprimés en PLA offre un positionnement précis du cube. Le calibrage individuel par bras, rendu possible par l'utilisation de dictionnaires de constantes dans le firmware, a permis de compenser les asymétries mécaniques inévitables dans un système d'impression 3D.

Sur le plan logiciel, l'architecture modulaire avec séparation PC/PICO et protocole Bluetooth bien défini a facilité le travail en équipe. La détection couleur par vote majoritaire et la résolution via l'algorithme Kociemba embarqué localement forment un pipeline robuste, capable de résoudre un cube mélangé en une vingtaine de secondes.

Ce projet nous a apporté de nombreux enseignements. Nous avons appris à travailler sur un système pluridisciplinaire où chaque partie dépend des autres, à gérer les contraintes du monde physique (tolérances mécaniques, variations d'éclairage, bruit électronique), et à nous organiser efficacement en équipe sur plusieurs mois.

En termes de perspectives d'amélioration, plusieurs pistes sont envisageables : l'utilisation de servomoteurs à retour de position pour une meilleure précision, l'intégration d'un éclairage contrôlé autour de la caméra pour une détection couleur plus stable, ou encore l'utilisation d'un modèle de vision par apprentissage automatique pour remplacer la détection HSV.

6. Bibliographie

Documentation technique des composants

- [1] Raspberry Pi Foundation. Raspberry Pi Pico Datasheet — RP2040 microcontroller. Disponible sur : <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>
- [2] NXP Semiconductors. MFRC522 — Standard performance MIFARE and NTAG frontend, Datasheet Rev. 3.9. 2016.
- [3] Solomon Systech. SSD1306 — 128x64 Dot Matrix OLED/PLED Segment/Common Driver with Controller. 2008.
- [4] TOGIALE. TJ-L5FCMXHTCSLCRGA5 — RGB LED Through-Hole Component. Fiche produit LCSC référence C2837020.
- [5] SHOU HA. TS665CJ — Tactile Push Button Switch. Fiche produit LCSC référence C393938.
- [6] HC-05 Bluetooth Serial Module — AT Commands Reference and Datasheet. Document technique fabricant.

Logiciels et outils de développement

- [7] EasyEDA — Logiciel de conception de schémas et PCB en ligne. JLCPCB. Disponible sur : <https://easyeda.com/>
- [8] Autodesk Fusion 360 — Logiciel de modélisation 3D paramétrique. Disponible sur : <https://www.autodesk.com/products/fusion-360/>
- [9] Visual Studio Code — Editeur de code source. Microsoft. Disponible sur : <https://code.visualstudio.com/>
- [10] MicroPico — Extension VS Code pour le développement MicroPython sur Raspberry Pi Pico. Disponible sur : <https://marketplace.visualstudio.com/items?itemName=paulober.pico-w-go>

Bibliothèques logicielles

- [11] Python Software Foundation. Python 3.13 Documentation. Disponible sur : <https://docs.python.org/3/>
- [12] MicroPython. MicroPython Documentation for RP2040. Disponible sur : <https://docs.micropython.org/>
- [13] OpenCV Team. OpenCV 4.x Documentation. Disponible sur : <https://docs.opencv.org/>
- [14] PyPI. pyserial 3.5 — Python Serial Port Extension. Disponible sur : <https://pypi.org/project/pyserial/>
- [15] PyPI. kociemba 1.2.1 — Python wrapper for Herbert Kociemba's two-phase algorithm. Disponible sur : <https://pypi.org/project/kociemba/>
- [16] PyPI. numpy — Fundamental package for scientific computing with Python. Disponible sur : <https://numpy.org/>
- [17] Python Standard Library. tkinter — Python Interface to Tcl/Tk. Disponible sur : <https://docs.python.org/3/library/tkinter.html>

- [18] PyPI. Pillow 10.x — Python Imaging Library (PIL Fork). Disponible sur : <https://pypi.org/project/pillow/>

Algorithme de résolution

- [19] Kociemba, H. Two-Phase Algorithm for solving Rubik's Cube. Documentation officielle. Disponible sur : <http://kociemba.org/cube.htm>
- [20] Rokicki, T., Kociemba, H., Davidson, M., Dethridge, J. God's Number is 20. 2010. Disponible sur : <https://www.cube20.org/>
- [21] Rui Santos & Sara Santos. Raspberry Pi Pico Servo Motor with MicroPython. Random Nerd Tutorials. Disponible sur : <https://randomnerdtutorials.com/raspberry-pi-pico-servo-motor-micropython/>

7. Annexes

Annexe A — Liste complète des composants

Composant	Référence LCSC	Qté	Rôle
Raspberry Pi Pico W	—	1	Microcontrôleur principal
Module Bluetooth HC-05	C40877	1	Communication PC-PICO
Lecteur RFID MFRC522	C27438	1	Identification badge
Ecran OLED SSD1306 0,96"	C2718488	1	Affichage état robot
LED RGB TJ-L5FCMXHTCSLRGB	C2837020	1	Indicateur visuel d'état
Servomoteur MG90S	C124376	8	4 rotations + 4 pinces (22,5×12×35,5mm, 13,4g, 1,8 kg/cm)
Bouton TS665CJ	C393938	1	Contrôle physique
Connecteur DC-005-5A-2.0	C381116	1	Alimentation servomoteurs 5V
Résistances Re_P14	—	3	Protection GPIO
Caméra USB	—	1	Détection couleurs des faces
PCB sur mesure (EasyEDA)	81,15x88,14mm	1	Support de tous les composants
Base laser	—	1	Structure principale du robot
Pièces imprimées PLA	—	INSERER NB	Bras, pinces, coupleurs, engrenages

Tableau A1 — Liste complète des composants avec références LCSC

Annexe B — Répartition des tâches

Membre	Fichiers et tâches réalisés
EL YAZAMI Yassine	PC/bluetooth/bt_uart.py — PC/ui/interface.py — PICO/bluetooth.py — PICO/oled_display.py — PICO/rfid_reader.py Interface graphique Tkinter, communication Bluetooth PC et PICO, affichage OLED, gestion RFID
MAZID Mohamed	PC/graphique/cube_view.py — PC/vision/color_detection.py — PC/solver/cube_solver.py — PC/vision/camera.py Détection couleur HSV, widget CubeView, résolution Kociemba, pipeline caméra

ELHAMAYDA Amer	PICO/main.py — PICO/servo_controller.py — PICO/servo.py — PICO/led_controller.py — PICO/state.py Firmware principal PICO, contrôle servomoteurs, calibrage, LED, machine à états
----------------	--

Tableau B1 — Répartition des tâches par membre de l'équipe

Annexe C — Diagramme de Gantt

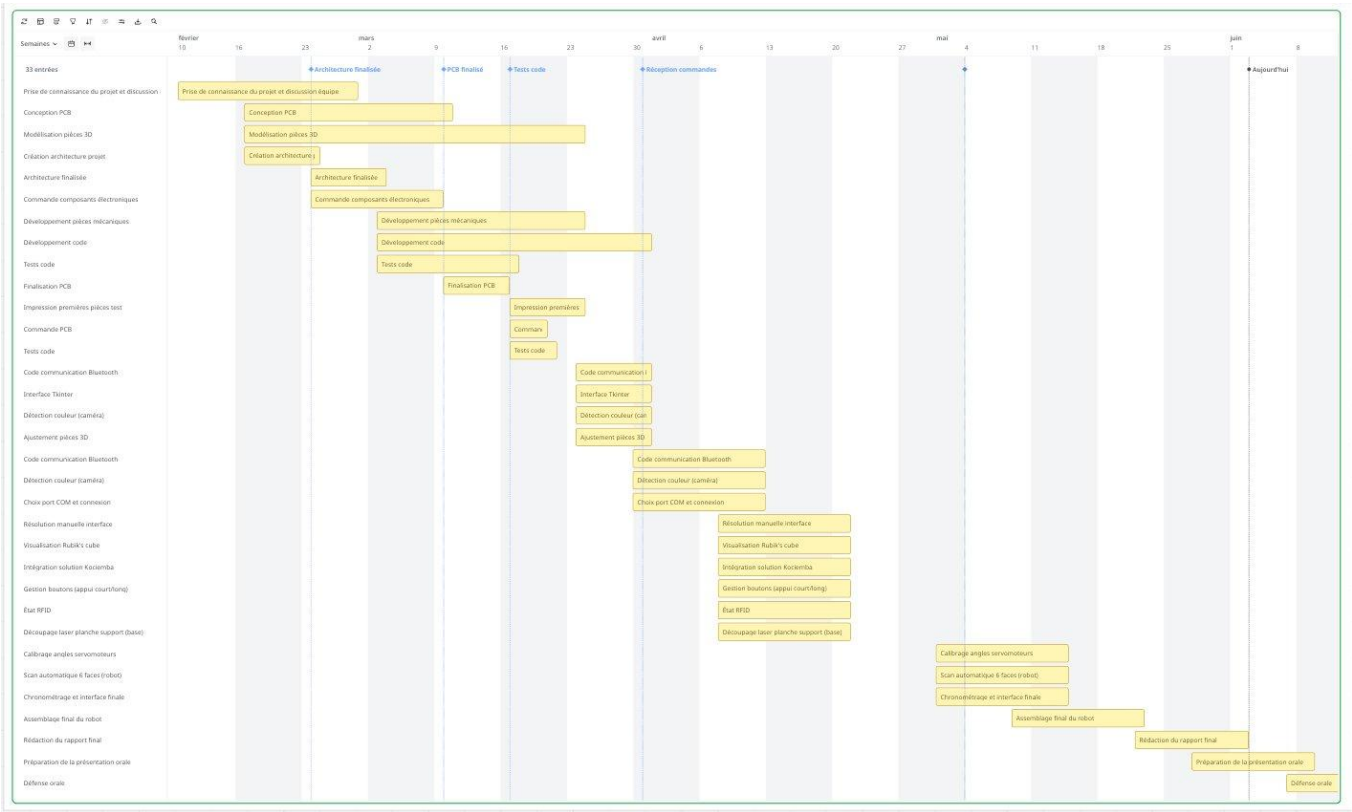


Figure C1 — Diagramme de Gantt complet du projet (février — juin 2026)

Annexe D — Informations PCB

Re...	Nom bom	Designator	Empreinte	Qu...	Pièce fabricant	Fabricant	Distributeur	Pièce du distributeur	Prix
1	DC-005-5...	DC1	DC-IN-TH_DC...	1	DC-005-5A-2.0	XKB Conn...	LCSC	C381116 Assign LCSC Part#	0.257
2	210S-1*2P...	H1	HDR-TH_2P-P...	1	B-2100S02P-A110	Ckmtw(世...	LCSC	C124375 Assign LCSC Part#	0.03
3	TJ-L5FCM...	LED1	LED-TH_4P-B...	1	TJ-L5FCMXHTCSLC...	TOGIALE...	LCSC	C2837020 Assign LCSC Part#	0.132
4	Motor_Servo	M1,M2,M3...	HDR-M-2.54_...	8	B-2100S03P-A110	Ckmtw(世...	LCSC	C124376 Assign LCSC Part#	0.034
5	0.96OLED...	OLED1	0.96OLED_4P	1	2.54-1*4P母	BOOMELE...	LCSC	C2718488 Assign LCSC Part#	0.078
6	Header-Fe...	P1,P2	HDR-TH_20P...	2	2.54-1*20P母	BOOMELE...	LCSC	C50984 Assign LCSC Part#	0.22
7	Re_P14	R1,R2,R3	RESISTOR AX...	3				Assign LCSC Part#	
8	TS665CJ	SW1	SW-TH_4P-L6...	1	TS665CJ	SHOU HA...	LCSC	C393938 Assign LCSC Part#	0.019
9	HC-05-6P	U1	HDR-TH_6P-P...	1	2.54-1*6P母	BOOMELE...	LCSC	C40877 Assign LCSC Part#	0.088
10	MFRCS22	U3	MFRCS22	1	2.54-1*8P母环保	BOOMELE...	LCSC	C27438 Assign LCSC Part#	0.107

Figure D1 — Caractéristiques techniques du PCB (EasyEDA)

Annexe E — Démonstration vidéo

Une vidéo de démonstration a été réalisée. Elle présente la résolution automatique complète du Rubik's Cube par le robot, de la reconnaissance du badge RFID jusqu'à la résolution finale par les servomoteurs.

Lien vers la vidéo de démonstration : [Vidéo — Résolution automatique du Rubik's Cube par les servomoteurs](#)